AI-Powered Fire Prediction Spread and Containment System

Technical Installation Manual

Panic! At The Kernel
Client: EPI-USE Africa

September 2026



Full Name	Student Number
Megan Lai	u23608821
Inge Keyser	u23544563
Janri du Toit	u23632730
Ryan Lynn	u23541912
Marco de Wit	u22811975

Contents

1 Introduction **3**

2 Overview **3**

3 Simple User Installation (PWA) **3**
3.1 Prerequisites . . . 3
3.2 Installing on Desktop (Chrome / Edge) . . . 3
3.3 Installing on Android . . . 3
3.4 Installing on IOS(Safari . . . 4

4 Technical Installation **4**
4.1 Prerequisites . . . 4
4.2 Installing Docker . . . 4

5 Local Development Installation **5**
5.1 5. Stopping the Stack . . . 6

6 AI Inference Worker Installation **7**
6.1 Prerequisites . . . 7
6.2 Setup . . . 7
6.3 Shared Artifact Store . . . 7
6.4 Install as a Background Service . . . 8

7 System Requirements **8**
7.1 Running FireAway as a User (Browser / PWA) . . . 8
7.2 Running the Full Stack Locally (Docker) . . . 8
7.3 Running the AI Inference Worker . . . 8

8 General Usage **9**

9 Troubleshooting **9**

10 Change Log **10**

1 Introduction

FireAway is a web-based application and installable Progressive Web App (PWA) built to help communities and emergency responders, in South Africa, to stay ahead of wildfires. It gives the public a live map of active fires and a simple way to report new ones, while giving authorised firefighters access to an AI-driven fire spread model that aids in planning containment strategies.

2 Overview

The application consists of the following components:

- **Frontend Web App/PWA:** Built with Next.js (React) amd TypeScript. Runs in the browser and installs to a device's home screen or desktop like a native app. Supports offline use: the live fire ap and previously loaded data remain viewable without a connection and fire reports created offline are queued and automatically synced once connectivity returns.
- **Backend:** Built with Python(3.12) this runs on the server hosting the deployment and holds all the logic for the database, services, and endpoints.
- **AWS worker:** There is an AWS job queue set-up to accept workers and allow them to poll the deployment for fire simulation requests. This implementation allows the load of running the simulation to be handled by caple CUDA devices and allows for scaling compute power.

3 Simple User Installation (PWA)

FireAway is a Progressive Web App, so no app store or installer file is required. Visiting the deployed site in a supported browser is enough to use it, and it can optionally be installed so it behaves like a native app with its own icon and window.

3.1 Prerequisites

- A stable internet connection.
- A modern browser: Google Chrome, Microsoft Edge, or Safari (desktop or mobile)

3.2 Installing on Desktop (Chrome / Edge)

1. Open your browser and navigate to the FireAway site: `https://fireaway.csml.co.za`
2. Log in with your credentials, or register a new account.
3. Click install icon on the address bar (a monitor-with-arrow), or open browser menu and select "Install FireAway..." / "Apps >Install the site as an app"
4. Confirm the install prompt. FireAway will now open in its own window and appear in your Start Menu / Applications folder.

3.3 Installing on Android

1. Open your browser and navigate to the FireAway site: `https://fireaway.csml.co.za`
2. Select "Install app" or "Add to Home screen"
3. Confirm the install prompt. A FireAway icon will be added to your home screen, and the app will launch full-screen without browser controls.

3.4 Installing on IOS(Safari

1. Open Safari and navigate to the FireAway site: `https://fireaway.csml.co.za`
2. Tap the share icon at the bottom of the screen.
3. Tap "Add" in the top right corner. A FireAway icon will appear on your home screen.

Note: *Installing the PWA is optional. FireAway will already work in any modern browser. Installation simply gives an app-like icon and window*

4 Technical Installation

This section is for developers and technical users who want to run the full FireAway stack locally, or contribute to the project.

4.1 Prerequisites

- Git
- Docker Engine and the Docker Compose plugin (Docker Desktop on Windows/macOS, or Docker Engine + docker-compose-plugin on Linux)
- Node.js 22.x and Yarn (via Corepack) — only required if running the frontend outside Docker
- Python 3.12 — only required if running the backend outside Docker
- At least 4 GB of free RAM and 5 GB of free disk space for the containerised stack

Note: *Docker is the supported and recommended way to run FireAway locally. It installs and configures Postgres, MinIO, the backend, and the frontend together, so the Node.js and Python prerequisites above are not needed for a standard install.*

4.2 Installing Docker

Linux (Ubuntu/Debian-based

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER
```

Log out and back in for the group change to take effect, then verify the install:

```
docker --version
docker compose version
```

Windows / macOS

- Download and install Docker Desktop from the official Docker website.
- Launch Docker Desktop and ensure it shows "Engine running" before continuing.

5 Local Development Installation

1. **Clone the repository**

```
       git clone https://github.com/[org]/AI-Powered-Fire-Spread-Prediction-and-
           Containment-System.git
cd AI-Powered-Fire-Spread-Prediction-and-Containment-System
```

2. **Configure environment variables**

   Create a `.env` file in project root (copy from `.env.example` if one is provided) and set the following variables:

Variable	Description

`POSTGRES_USER`
`POSTGRES_PASSWORD`
`POSTGRES_DB`
	Credentials and name for the local Postgres database.

`DB_HOST`
`DB_PORT`
`DB_USER`
`DB_PASSWORD`
`DB_NAME`
	Connection details used by the backend to reach Postgres.

`MINIO_ROOT_USER` `MINIO_ROOT_PASSWORD`	Admin credentials for the MinIO object storage console.
`MINIO_ENDPOINT` `MINIO_PUBLIC_ENDPOINT`	Internal and browser-facing MinIO endpoints.

`MINIO_ACCESS_KEY`
`MINIO_SECRET_KEY`
`MINIO_BUCKET`
	Credentials and bucket name the backend uses to store images.

`BACKEND_HOST` `BACKEND_PORT`	Host and port the backend API listens on (default `backend` / `8000`).
`NEXT_PUBLIC_API_URL` `BACKEND_INTERNAL_URL`	URLs the frontend uses to reach the backend, from the browser and from within Docker.
`JWT_SECRET_KEY`	Secret key used to sign authentication tokens. Generate with: `openssl rand -hex 32`
`NEXT_PUBLIC_MAPBOX_TOKEN`	Mapbox access token, used to render maps in the dashboard.

Variable	Description
PGADMIN_DEFAULT_EMAIL PGADMIN_DEFAULT_PASSWORD	Login for the optional pgAdmin database UI.
AWS_REGION	AWS region hosting the asynchronous SQS queues
INFERENCE_QUEUE_URL	SQS queue URL where the backend enqueues fire simulation jobs for GPU workers.
RESULTS_QUEUE_URL	SQS queue URL where the GPU worker publishes completion metadata and result pointers.
WORKER_ID	Identifier for the worker node publishing inference results
ARTIFACTS_ROOT	Base path for shared model checkpoints and simulation output files.
HEARTBEAT_FILE	Local system path touched on every polling loop for systemd health checks.

```
docker compose up -d --build
```

This builds and starts the following containers: `postgres`, `minio`, `backend` (FastAPI/uvicorn), and `frontend` (Next.js). The first build can take a few minutes while dependencies install.

Check that everything started correctly:

```
docker compose ps
docker compose logs -f backend
docker compose logs -f frontend
```

Service	URL	Notes
Frontend (Web App / PWA)	`http://localhost:3000`	Main FireAway dashboard
Backend API	`http://localhost:8000`	REST API and interactive docs at `/docs`
MinIO Console	`http://localhost:9001`	Object storage admin UI
pgAdmin	`http://localhost:5050`	Optional Postgres admin UI, if enabled

Note: *Register a new account or log in from* `http://localhost:3000`*. Because FireAway is a PWA, you can also install it from the browser at this local address using the same steps as the Simple User Installation section.*

5.1 5. Stopping the Stack

To shut down the running containers:

```
docker compose down
```

Add `-v` to also remove the Postgres and MinIO data volumes if you want a completely clean reset:

```
docker compose down -v
```

6 AI Inference Worker Installation

The AI inference worker runs the fire-spread prediction model on a CUDA-capable GPU and processes jobs from a message queue. This component is needed for the usage of the inference of the backend especially if you want to the inference yourself.

6.1 Prerequisites

- A CUDA-capable NVIDIA GPU
- A Linux distribution (any modern distro; Docker is not required for this component)
- Python 3.10 or higher
- AWS credentials with access to the project's SQS queues (provided by the project maintainer)

6.2 Setup

```
sudo mkdir -p /opt/fire-system/gpu_worker
sudo chown $USER /opt/fire-system/gpu_worker
cd /opt/fire-system/gpu_worker

python3 -m venv venv

# Check your CUDA version first
nvidia-smi # note the CUDA version shown top-right

# Install the CUDA-matched build of torch BEFORE requirements.txt
# Replace cu121 with your actual CUDA version (cu118, cu121, cu124, etc.)
venv/bin/pip install torch --index-url https://download.pytorch.org/whl/cu121

# Then install the remaining dependencies
venv/bin/pip install -r requirements.txt

cp .env.example .env
nano .env # fill in the queue URLs

mkdir -p ~/.aws
nano ~/.aws/credentials # paste the AWS access keys provided to you
chmod 600 ~/.aws/credentials
```

6.3 Shared Artifact Store

Before the worker can load model checkpoints, mount the shared S3 artifact store using `rclone` so that all inference machines use the same model version:

```
# See the rclone setup steps in the deployment guide
# Mounts to: /mnt/fire-system-artifacts/
```

6.4 Install as a Background Service

```
sudo cp fire-worker.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now fire-worker
```

Check that the worker is running and polling the queue:

```
sudo systemctl status fire-worker
journalctl -u fire-worker -f
```

Note: *The worker writes a heartbeat file (default `/tmp/gpu_worker_heartbeat`) after every poll cycle. If this file has not updated in over two minutes, the worker is stuck or has crashed.*

7 System Requirements

7.1 Running FireAway as a User (Browser / PWA)

Requirement	Minimum
Browser	Latest Chrome, Edge, or Safari
OS	Windows 10+, macOS 12+, Android 8+, or iOS 15+
Internet	Stable broadband or mobile data connection

7.2 Running the Full Stack Locally (Docker)

Component	Minimum	Recommended
CPU	4 cores	8 cores
RAM	8 GB	16 GB
Disk Space	10 GB free	25 GB free (SSD)
OS	Windows 11, macOS 12+, or Linux	Windows 11 or Linux

7.3 Running the AI Inference Worker

Component	Minimum	Recommended
GPU	CUDA-capable NVIDIA GPU, 6 GB VRAM	NVIDIA GPU, 12 GB+ VRAM
CPU	4 cores	8 cores
RAM	16 GB	32 GB
OS	Linux (any modern distribution)	Linux (any modern distribution)

8 General Usage

After installing and launching FireAway(either in a browser or as an installed PWA), log in or register an account to access the dashboard. From there you can view the live map, monitor fire risk and predicted spread for your area, report incidents, and run the simulation.

For more information refer to the FireAway help manual.

9 Troubleshooting

- Port already in use: stop any other services using ports 3000, 8000, 9000/9001, or 5432, or change the mapped ports in docker-compose.yml.
- Containers fail to start: run docker compose logs ¡service¿ to view the error, and confirm all required variables in .env are set.
- Frontend cannot reach backend: confirm `NEXT_PUBLIC_API_URL` matches the address the browser can actually reach (http://localhost:8000 for local development).
- Out of disk space during build: run docker system prune -a to clear unused images and build cache.

10 Change Log

Date	Version	Change
September 2026	1.0	Initial version (Demo 3)